

Cloud Spanner Game Trading Post

來源：<https://codelabs.developers.google.com/codelabs/cloud-spanner-game-trade-post?hl=zh-tw>

步驟數：8

在本程式碼研究室中，您將實作商品服務和交易後服務，以便與 Cloud Spanner 搭配使用。

目錄

1. [簡介](#)
2. [設定和需求](#)
3. [部署項目服務](#)
4. [部署 tradepost 服務](#)
5. [開始交易](#)
6. [擷取交易統計資料](#)
7. [正在清除所用資源](#)
8. [恭喜！](#)

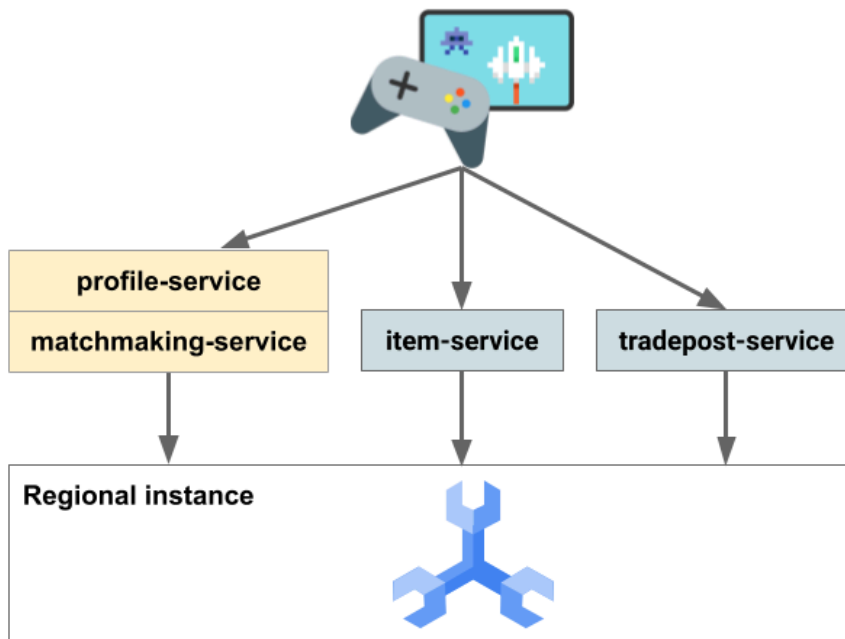
1. 簡介

Cloud Spanner 是全代管的關聯式資料庫服務，可水平擴充並在全球各地部署，提供 ACID 交易和 SQL 語意，同時維持效能和高可用性。

因此，如果遊戲架構要啟用全球玩家群，或擔心資料一致性問題，Spanner 就是絕佳選擇

在本實驗室中，您會建立兩項與區域 Spanner 資料庫互動的 Go 服務，讓玩家取得物品和金錢 (`item-service`)，然後在交易平台列出物品供其他玩家購買 (`trade-post-service`)。

本實驗室需要依賴「[Cloud Spanner Getting Started with Games Development](#)」程式碼研究室，使用 `profile-service` 和 `matchmaking-service` 產生玩家和遊戲。



Google

Confidential + Proprietary

接著，您將運用 Python 負載架構 [Locust.io](#) 產生資料，模擬玩家在「遊戲」過程中取得金錢和物品。玩家可以在交易站上架商品，其他有足夠資金的玩家就能購買。

您也會查詢 Spanner，判斷玩家的帳戶餘額和商品數量，以及有關未結或已完成交易訂單的一些統計資料。

最後，您將清除在本實驗室中建立的資源。

建構項目

本實驗室的學習內容包括：

- 重複使用「[開始使用 Cloud Spanner 開發遊戲](#)」中的 Spanner 執行個體。
- 部署以 Go 編寫的 Item 服務，處理玩家取得物品和金錢的作業
- 部署以 Go 編寫的交易站服務，模擬玩家列出待售物品，以及其他玩家購買這些物品。

課程內容

- 如何使用讀取/寫入交易，確保資料變更的一致性
- 如何運用 DML 和 Spanner 變異來修改資料

軟硬體需求

- 已連結至帳單帳戶的 Google Cloud 專案。
 - 網路瀏覽器，例如 [Chrome](#) 或 [Firefox](#)。
 - 先前已完成「[Cloud Spanner Getting Started with Games Development](#)」，但未執行清除步驟。
-

2. 設定和需求

完成「開始使用 Cloud Spanner 開發遊戲」程式碼研究室

完成「[開始使用 Cloud Spanner 開發遊戲](#)」程式碼研究室。您必須啟用這項功能，才能取得球員和賽事資料集。玩家和遊戲需要取得物品和金錢，然後用來在交易站刊登待售物品和購買物品。

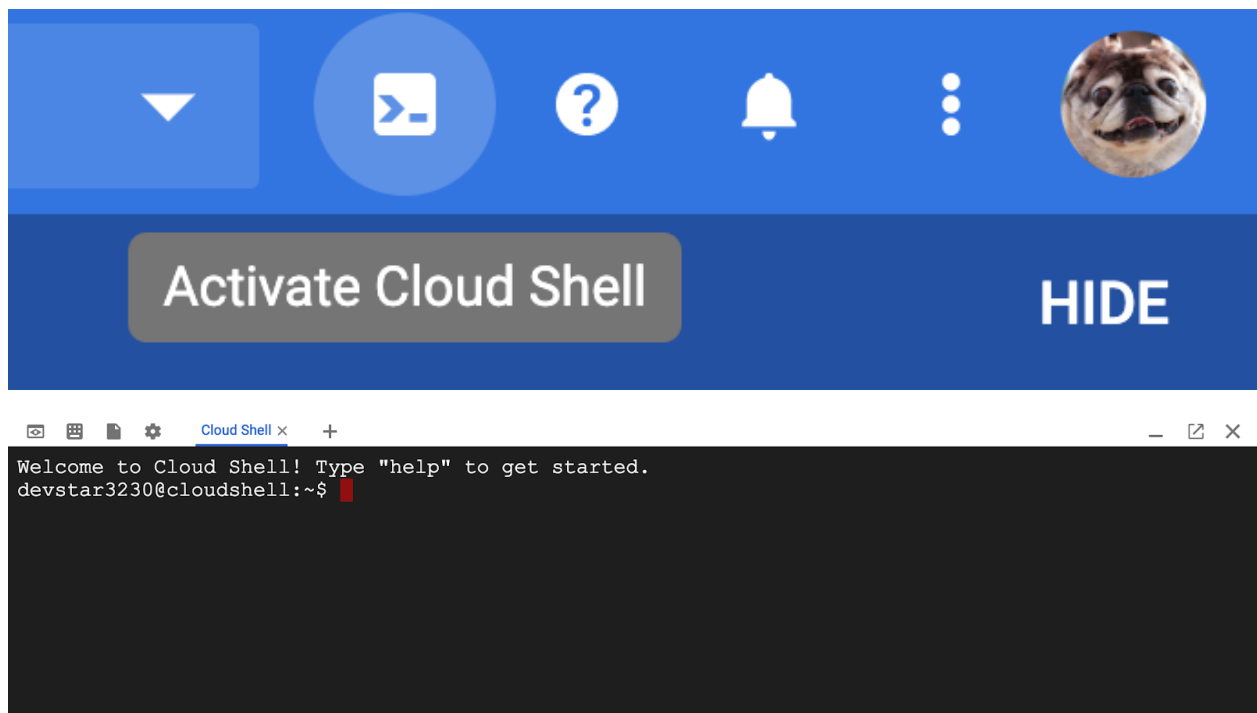
注意：請勿執行依附元件程式碼研究室的清除步驟。否則您必須重新完成程式碼研究室。

在 Cloud Shell 中設定環境變數

按一下「啟用 Cloud Shell」圖示



開啟 Cloud Shell (由於您先前已執行這項操作，佈建並連線至環境的作業應該只需要幾分鐘)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的 PROJECT_ID。

在 Cloud Shell 中設定 SPANNER 環境變數

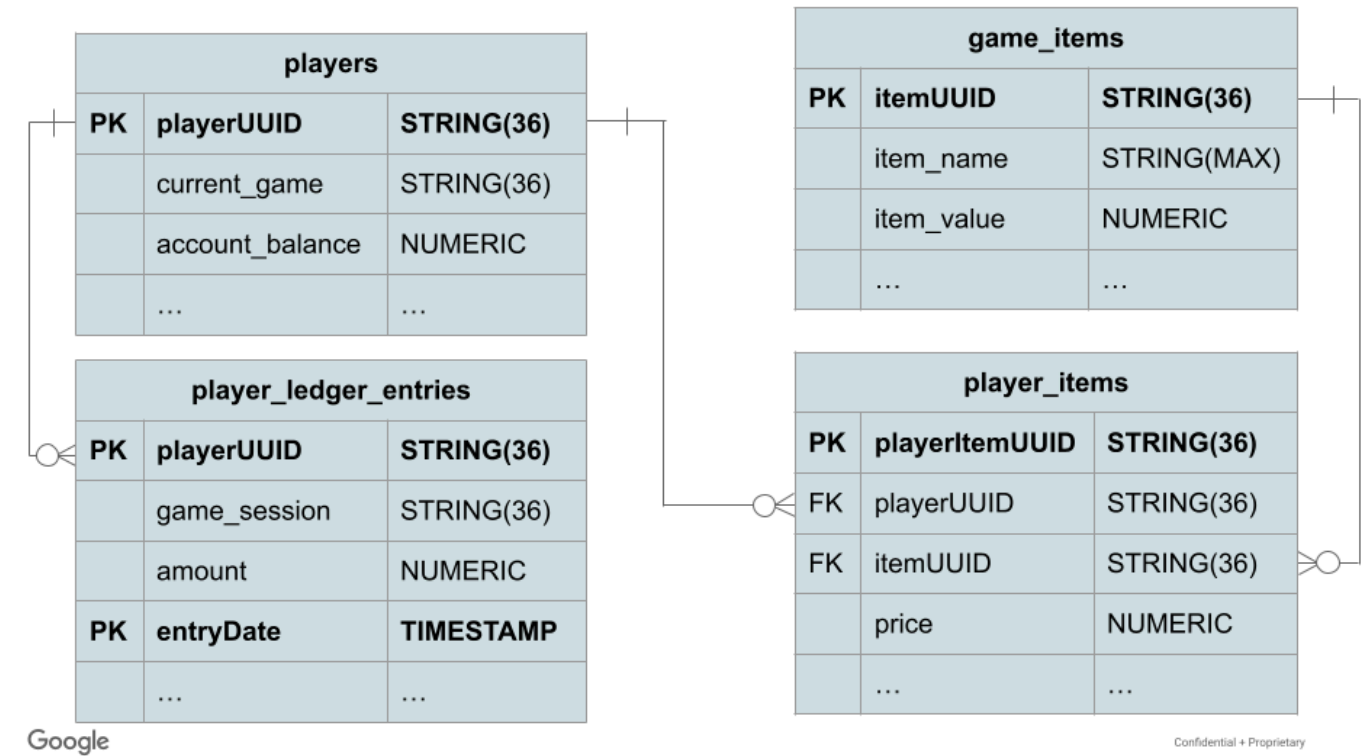
```
export SPANNER_PROJECT_ID=$GOOGLE_CLOUD_PROJECT
export SPANNER_INSTANCE_ID=cloudspanner-gaming
export SPANNER_DATABASE_ID=sample-game
```

警告：請務必將 <INSTANCE_ID> 替換為實際的執行個體 ID。

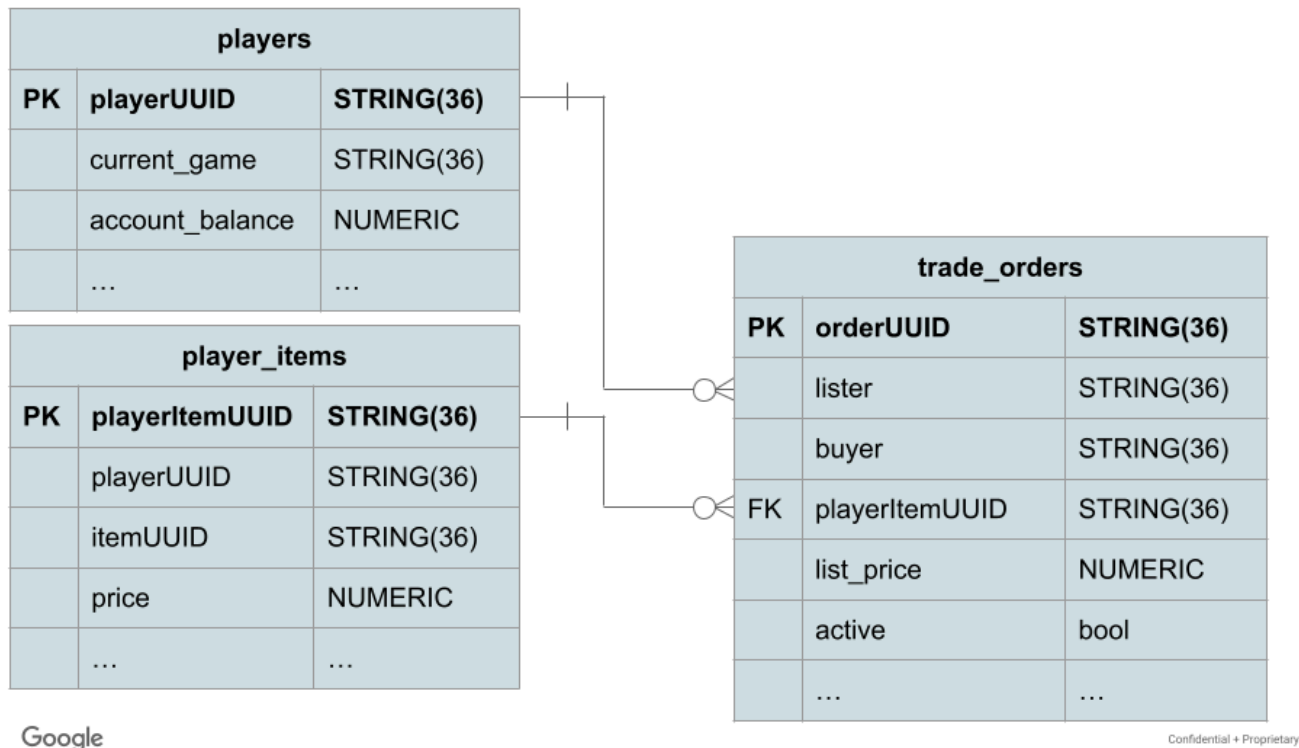
建立結構定義

資料庫建立完成後，您可以在 `sample-game` 資料庫中定義結構定義。

本實驗室會建立四個新資料表：`game_items`、`player_items`、`player_ledger_entries` 和 `trade_orders`。



播放器項目關係



交易訂單關係

遊戲道具會新增至 `game_items` 表格，然後玩家就能取得。`player_items` 資料表同時具有 `itemUUID` 和 `playerUUID` 的外部鍵，確保玩家只會取得有效項目。

`player_ledger_entries` 表格會追蹤玩家帳戶餘額的所有金額變動。你可以從戰利品中取得金錢，或在交易站販售物品。

最後，`trade_orders` 資料表用於處理發布的賣單，以及買家完成這些訂單。

如要建立結構定義，請在 Cloud 控制台中按一下 `Write DDL` 按鈕：

All instances >

INSTANCE

cloudspanner-gaming: Overview >

GOOGLE STANDARD SQL DATABASE

sample-game: Overview

WRITE DDL

Overview

Database name	sample-game
Database dialect	Google Standard SQL
Encryption type	Google-managed
Schema updates	No recent updates

在這裡，您將輸入 [schema/trading.sql](#) 檔案中的結構定義：

```

CREATE TABLE game_items
(
    itemUUID STRING(36) NOT NULL,
    item_name STRING(MAX) NOT NULL,
    item_value NUMERIC NOT NULL,
    available_time TIMESTAMP NOT NULL,
    duration int64
)PRIMARY KEY (itemUUID);

CREATE TABLE player_items
(
    playerItemUUID STRING(36) NOT NULL,
    playerUUID STRING(36) NOT NULL,
    itemUUID STRING(36) NOT NULL,
    price NUMERIC NOT NULL,
    source STRING(MAX) NOT NULL,
    game_session STRING(36) NOT NULL,
    acquire_time TIMESTAMP NOT NULL DEFAULT (CURRENT_TIMESTAMP()),
    expires_time TIMESTAMP,
    visible BOOL NOT NULL DEFAULT(true),
    FOREIGN KEY (itemUUID) REFERENCES game_items (itemUUID),
    FOREIGN KEY (game_session) REFERENCES games (gameUUID)
) PRIMARY KEY (playerUUID, playerItemUUID),
    INTERLEAVE IN PARENT players ON DELETE CASCADE;

CREATE TABLE player_ledger_entries (
    playerUUID STRING(36) NOT NULL,
    source STRING(MAX) NOT NULL,
    game_session STRING(36) NOT NULL,
    amount NUMERIC NOT NULL,
    entryDate TIMESTAMP NOT NULL OPTIONS (allow_commit_timestamp=true),
    FOREIGN KEY (game_session) REFERENCES games (gameUUID)
) PRIMARY KEY (playerUUID, entryDate DESC),
    INTERLEAVE IN PARENT players ON DELETE CASCADE;

CREATE TABLE trade_orders
(
    orderUUID STRING(36) NOT NULL,
    lister STRING(36) NOT NULL,
    buyer STRING(36),
    playerItemUUID STRING(36) NOT NULL,
    trade_type STRING(5) NOT NULL,
    list_price NUMERIC NOT NULL,
    created TIMESTAMP NOT NULL DEFAULT (CURRENT_TIMESTAMP()),
    ended TIMESTAMP,
    expires TIMESTAMP NOT NULL DEFAULT (TIMESTAMP_ADD(CURRENT_TIMESTAMP(), interval 24 HOUR)),
    active BOOL NOT NULL DEFAULT (true),
    cancelled BOOL NOT NULL DEFAULT (false),
    filled BOOL NOT NULL DEFAULT (false),
    expired BOOL NOT NULL DEFAULT (false),
    FOREIGN KEY (playerItemUUID) REFERENCES player_items (playerItemUUID)
) PRIMARY KEY (orderUUID);

```

```
CREATE INDEX TradeItem ON trade_orders(playerItemUUID, active);
```

按一下「Submit」按鈕修改結構定義，並等待結構定義更新完成：

Overview

Database name	sample-game
Database dialect	Google Standard SQL
Encryption type	Google-managed
Schema updates	 Update completed To view all updates, go to Cloud Logging .

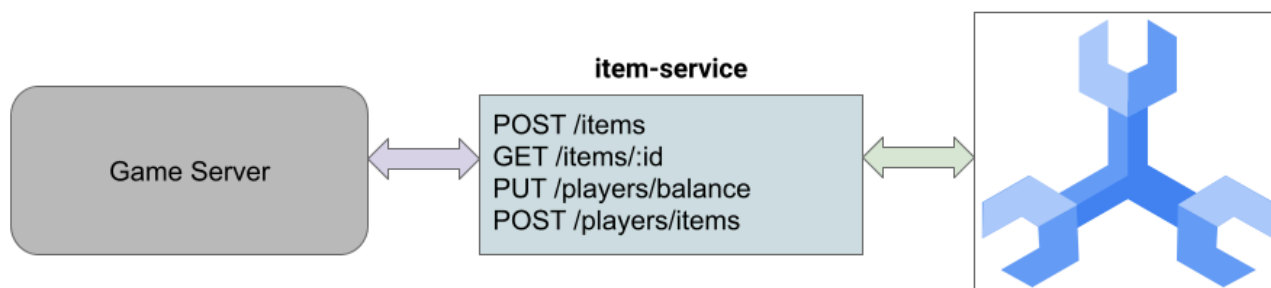
下一步

接著，您將部署項目服務。

3. 部署項目服務

服務總覽

項目服務是以 Go 撰寫的 REST API，採用 gin 架構。在這個 API 中，參與開放式遊戲的玩家可獲得金錢和物品。



Google

Confidential + Proprietary

`./src/golang/item-service/main.go` 檔案會設定下列端點，以便處理遊戲道具和取得道具的玩家。此外，玩家還可透過端點取得金錢。

```
func main() {
    configuration, _ := config.NewConfig()

    router := gin.Default()
    router.SetTrustedProxies(nil)
    router.Use(setSpannerConnection(configuration))

    router.GET("/items", getItemUUIDs)
    router.POST("/items", createItem)
    router.GET("/items/:id", getItem)
    router.PUT("/players/balance", updatePlayerBalance)
    router.GET("/players", getPlayer)
    router.POST("/players/items", addPlayerItem)

    router.Run(configuration.Server.URL())
}
```

設定及使用 Spanner 連線的方式，與先程式碼研究室中的 `profile-service` 和 `matchmaking-service` 完全相同。

項目服務會搭配 **`GameItem`**、**`Player`**、**`PlayerLedger`** 和 **`PlayerItem`** 使用，定義如下：

```

// models/game_items.go
type GameItem struct {
    ItemUUID      string    `json:"itemUUID"`
    Item_name     string    `json:"item_name"`
    Item_value    big.Rat   `json:"item_value"`
    Available_time time.Time `json:"available_time"`
    Duration      int64     `json:"duration"`
}

// models/players.go
type Player struct {
    PlayerUUID      string    `json:"playerUUID" binding:"required,uuid4"`
    Updated         time.Time `json:"updated"`
    Account_balance big.Rat   `json:"account_balance"`
    Current_game    string    `json:"current_game"`
}

type PlayerLedger struct {
    PlayerUUID string `json:"playerUUID" binding:"required,uuid4"`
    Amount     big.Rat `json:"amount"`
    Game_session string `json:"game_session"`
    Source     string `json:"source"`
}

// models/player_items.go
type PlayerItem struct {
    PlayerItemUUID string    `json:"playerItemUUID" binding:"omitempty,uuid4"`
    PlayerUUID      string    `json:"playerUUID" binding:"required,uuid4"`
    ItemUUID        string    `json:"itemUUID" binding:"required,uuid4"`
    Source          string    `json:"source" binding:"required"`
    Game_session    string    `json:"game_session" binding:"omitempty,uuid4"`
    Price           big.Rat   `json:"price"`
    AcquireTime     time.Time `json:"acquire_time"`
    ExpiresTime     spanner.NullTime `json:"expires_time"`
    Visible         bool      `json:"visible"`
}

```

首先，遊戲必須建立一些項目。為此，系統會呼叫 */items* 端點的 POST 要求。這是非常簡單的 DML 插入作業，會將資料插入 *game_items* 資料表。

```

// main.go
func createItem(c *gin.Context) {
    var item models.GameItem

    if err := c.BindJSON(&item); err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    ctx, client := getSpannerConnection(c)
    err := item.Create(ctx, client)
    if err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    c.IndentedJSON(http.StatusCreated, item.ItemUUID)
}

// models/game_items.go
func (i *GameItem) Create(ctx context.Context, client spanner.Client) error {
    // Initialize item values
    i.ItemUUID = generateUUID()

    if i.Available_time.IsZero() {
        i.Available_time = time.Now()
    }

    // insert into spanner
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        stmt := spanner.Statement{
            SQL: `INSERT game_items (itemUUID, item_name, item_value, available_time,
duration)
                VALUES (@itemUUID, @itemName, @itemValue, @availableTime, @duration)
            `,
            Params: map[string]interface{}{
                "itemUUID":      i.ItemUUID,
                "itemName":      i.Item_name,
                "itemValue":      i.Item_value,
                "availableTime": i.Available_time,
                "duration":       i.Duration,
            },
        }

        _, err := txn.Update(ctx, stmt)
        return err
    })

    if err != nil {
        return err
    }
}

```

```
// return empty error on success
return nil
}
```

如要取得項目，請呼叫 **/players/items** 端點的 POST 要求。這個端點的邏輯是擷取遊戲項目的目前值，以及玩家目前的遊戲工作階段。然後將適當資訊插入 *player_items* 資料表，指出項目取得來源和時間。

這會對應至下列函式：

```

// main.go
func addPlayerItem(c *gin.Context) {
    var playerItem models.PlayerItem

    if err := c.BindJSON(&playerItem); err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    ctx, client := getSpannerConnection(c)
    err := playerItem.Add(ctx, client)
    if err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    c.IndentedJSON(http.StatusCreated, playerItem)
}

// models/player_items.go
func (pi *PlayerItem) Add(ctx context.Context, client spanner.Client) error {
    // insert into spanner
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        // Get item price at time of transaction
        price, err := GetItemPrice(ctx, txn, pi.ItemUUID)
        if err != nil {
            return err
        }

        pi.Price = price

        // Get Game session
        session, err := GetPlayerSession(ctx, txn, pi.PlayerUUID)
        if err != nil {
            return err
        }

        pi.Game_session = session

        pi.PlayerItemUUID = generateUUID()

        // Insert
        cols := []string{"playerItemUUID", "playerUUID", "itemUUID", "price", "source",
"game_session"}

        txn.BufferWrite([]*spanner.Mutation{
            spanner.Insert("player_items", cols,
                []interface{}{pi.PlayerItemUUID, pi.PlayerUUID, pi.ItemUUID, pi.Price,
pi.Source, pi.Game_session}),
        })
    })
}

```

```
        return nil
    })

    if err != nil {
        return err
    }

    // return empty error on success
    return nil
}
```

如要讓玩家取得金錢，請呼叫 **/players/updatebalance** 端點的 PUT 要求。

這個端點的邏輯是在套用「amount」後更新玩家餘額，並在「player_ledger_entries」資料表中更新取得記錄。系統會修改玩家的 account_balance，然後傳回給呼叫端。DML 用於修改玩家和 player_ledger_entries。

這會對應至下列函式：

```

// main.go
func updatePlayerBalance(c *gin.Context) {
    var player models.Player
    var ledger models.PlayerLedger

    if err := c.BindJSON(&ledger); err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    ctx, client := getSpannerConnection(c)
    err := ledger.UpdateBalance(ctx, client, &player)
    if err != nil {
        c.AbortWithError(http.StatusBadRequest, err)
        return
    }

    type PlayerBalance struct {
        PlayerUUID, AccountBalance string
    }

    balance := PlayerBalance{PlayerUUID: player.PlayerUUID, AccountBalance:
player.Account_balance.FloatString(2)}
    c.IndentedJSON(http.StatusOK, balance)
}

// models/players.go
func (l *PlayerLedger) UpdateBalance(ctx context.Context, client spanner.Client, p *Player)
error {
    // Update balance with new amount
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        p.PlayerUUID = l.PlayerUUID
        stmt := spanner.Statement{
            SQL: `UPDATE players SET account_balance = (account_balance + @amount) WHERE
playerUUID = @playerUUID`,
            Params: map[string]interface{}{
                "amount":      l.Amount,
                "playerUUID": p.PlayerUUID,
            },
        }
        numRows, err := txn.Update(ctx, stmt)

        if err != nil {
            return err
        }

        // No rows modified. That's an error
        if numRows == 0 {
            errorMsg := fmt.Sprintf("Account balance for player '%s' could not be updated",
p.PlayerUUID)
            return errors.New(errorMsg)
        }
    })
}

```

```

    }

    // Get player's new balance (read after write)
    stmt = spanner.Statement{
        SQL: `SELECT account_balance, current_game FROM players WHERE playerUUID =
@playerUUID`,
        Params: map[string]interface{}{
            "playerUUID": p.PlayerUUID,
        },
    }
    iter := txn.Query(ctx, stmt)
    defer iter.Stop()
    for {
        row, err := iter.Next()
        if err == iterator.Done {
            break
        }
        if err != nil {
            return err
        }
        var accountBalance big.Rat
        var gameSession string

        if err := row.Columns(&accountBalance, &gameSession); err != nil {
            return err
        }
        p.Account_balance = accountBalance
        l.Game_session = gameSession
    }

    stmt = spanner.Statement{
        SQL: `INSERT INTO player_ledger_entries (playerUUID, amount, game_session, source,
entryDate)
        VALUES (@playerUUID, @amount, @game, @source, PENDING_COMMIT_TIMESTAMP())`,
        Params: map[string]interface{}{
            "playerUUID": l.PlayerUUID,
            "amount":     l.Amount,
            "game":        l.Game_session,
            "source":      l.Source,
        },
    }
    numRows, err = txn.Update(ctx, stmt)
    if err != nil {
        return err
    }

    return nil
})

if err != nil {
    return err
}

```

```
    return nil
}
```

根據預設，服務會使用環境變數設定。請參閱 [./src/golang/item-service/config/config.go](#) 檔案的相關章節。

```
func NewConfig() (Config, error) {
    *snip*
    // Server defaults
    viper.SetDefault("server.host", "localhost")
    viper.SetDefault("server.port", 8082)

    // Bind environment variable override
    viper.BindEnv("server.host", "SERVICE_HOST")
    viper.BindEnv("server.port", "SERVICE_PORT")
    viper.BindEnv("spanner.project_id", "SPANNER_PROJECT_ID")
    viper.BindEnv("spanner.instance_id", "SPANNER_INSTANCE_ID")
    viper.BindEnv("spanner.database_id", "SPANNER_DATABASE_ID")

    *snip*

    return c, nil
}
```

您可以看到，預設行為是在 *localhost:8082* 上執行服務。

有了這些資訊，就可以執行服務了。

執行服務

執行服務會下載依附元件，並在通訊埠 **8082** 上建立服務：

```
cd ~/spanner-gaming-sample/src/golang/item-service
go run . &
```

注意：下載 golang 依附元件可能需要幾分鐘。

指令輸出：

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.
```

```
[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:    export GIN_MODE=release
- using code:   gin.SetMode(gin.ReleaseMode)
```

```
[GIN-debug] GET    /items                --> main.getItemUUIDs (4 handlers)
[GIN-debug] POST   /items                --> main.createItem (4 handlers)
[GIN-debug] GET    /items/:id            --> main.getItem (4 handlers)
[GIN-debug] PUT    /players/balance      --> main.updatePlayerBalance (4 handlers)
[GIN-debug] GET    /players              --> main.getPlayer (4 handlers)
[GIN-debug] POST   /players/items        --> main.addPlayerItem (4 handlers)
[GIN-debug] Listening and serving HTTP on localhost:8082
```

發出 curl 指令來建立項目，藉此測試服務：

```
curl http://localhost:8082/items \
  --include \
  --header "Content-Type: application/json" \
  --request "POST" \
  --data '{"item_name": "test_item", "item_value": "3.14"}'
```

指令輸出：

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: <Date>
Content-Length: 38

"aecde380-0a79-48c0-ab5d-0da675d3412c"
```

注意：每個要求的輸出 UUID 都會不同。

接著，您希望玩家取得這項物品。如要這麼做，您需要 *ItemUUID* 和 *PlayerUUID*。*ItemUUID* 是上一個指令的輸出內容。在本範例中，這個值為 `aecde380-0a79-48c0-ab5d-0da675d3412c`。

如要取得 *PlayerUUID*，請呼叫 *GET /players* 端點：

```
curl http://localhost:8082/players
```

指令輸出：

```
{
  "playerUUID": "b74cc194-87b0-4a55-a67f-0f0742ef6352",
  "updated": "0001-01-01T00:00:00Z",
  "account_balance": {},
  "current_game": "7b97fa85-5658-4ded-a962-4c09269a0a79"
}
```

注意：每次要求生成的內容都會不一樣。

如要讓玩家取得物品，請向 *POST /players/items* 端點提出要求：

```
curl http://localhost:8082/players/items \
  --include \
  --header "Content-Type: application/json" \
  --request "POST" \
  --data '{"playerUUID": "b74cc194-87b0-4a55-a67f-0f0742ef6352", "itemUUID": "109ec745-9906-402b-9d03-ca7153a10312", "source": "loot"}'
```

注意：請從上述輸出內容中，代入 *ItemUUID* 和 *PlayerUUID* 的值。

指令輸出：

```
Content-Type: application/json; charset=utf-8
Date: <Date>
Content-Length: 369

{
  "playerItemUUID": "a42b1899-4509-4fce-9958-265d2a2838a0",
  "playerUUID": "b74cc194-87b0-4a55-a67f-0f0742ef6352",
  "itemUUID": "109ec745-9906-402b-9d03-ca7153a10312",
  "source": "loot",
  "game_session": "7b97fa85-5658-4ded-a962-4c09269a0a79",
  "price": {},
  "acquire_time": "0001-01-01T00:00:00Z",
  "expires_time": null,
  "visible": false
}
```

注意：每次要求生成的內容都會不一樣。

摘要

在這個步驟中，您部署了可建立遊戲物品的物品服務，並指派玩家開啟遊戲，以便取得金錢和遊戲物品。

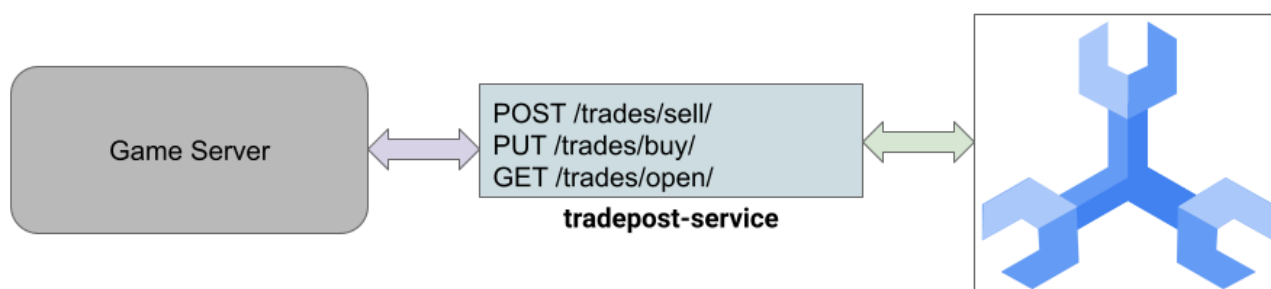
後續步驟

在下一個步驟中，您將部署 `trade-post` 服務。

4. 部署 tradepost 服務

服務總覽

tradepost 服務是以 Go 撰寫的 REST API，採用 gin 架構。在這個 API 中，玩家物品會發布至銷售。遊戲玩家接著就能取得公開交易，並在有足夠資金時購買商品。



Google

Confidential + Proprietary

tradepost 服務的 `./src/golang/tradepost-service/main.go` 檔案與其他服務的設定和程式碼類似，因此在此不重複說明。這項服務會公開下列幾個端點：

```
func main() {
    configuration, _ := config.NewConfig()

    router := gin.Default()
    router.SetTrustedProxies(nil)

    router.Use(setSpannerConnection(configuration))

    router.GET("/trades/player_items", getPlayerItem)
    router.POST("/trades/sell", createOrder)
    router.GET("/trades/open", getOpenOrder)
    router.PUT("/trades/buy", purchaseOrder)

    router.Run(configuration.Server.URL())
}
```

這項服務提供 **TradeOrder** 結構體，以及 **GameItem**、**PlayerItem**、**Player** 和 **PlayerLedger** 結構體的必要結構體：

```

type TradeOrder struct {
    OrderUUID      string      `json:"orderUUID" binding:"omitempty,uuid4"~`
    Lister          string      `json:"lister" binding:"omitempty,uuid4"~`
    Buyer           string      `json:"buyer" binding:"omitempty,uuid4"~`
    PlayerItemUUID string      `json:"playerItemUUID" binding:"omitempty,uuid4"~`
    TradeType       string      `json:"trade_type"~`
    ListPrice       big.Rat     `json:"list_price" spanner:"list_price"~`
    Created          time.Time   `json:"created"~`
    Ended           spanner.NullTime `json:"ended"~`
    Expires         time.Time   `json:"expires"~`
    Active          bool        `json:"active"~`
    Cancelled       bool        `json:"cancelled"~`
    Filled          bool        `json:"filled"~`
    Expired         bool        `json:"expired"~`
}

type GameItem struct {
    ItemUUID      string      `json:"itemUUID"~`
    ItemName      string      `json:"item_name"~`
    ItemValue     big.Rat     `json:"item_value"~`
    AvailableTime time.Time   `json:"available_time"~`
    Duration      int64       `json:"duration"~`
}

type PlayerItem struct {
    PlayerItemUUID string      `json:"playerItemUUID" binding:"omitempty,uuid4"~`
    PlayerUUID      string      `json:"playerUUID" binding:"required,uuid4"~`
    ItemUUID        string      `json:"itemUUID" binding:"required,uuid4"~`
    Source          string      `json:"source"~`
    GameSession     string      `json:"game_session" binding:"omitempty,uuid4"~`
    Price           big.Rat     `json:"price"~`
    AcquireTime     time.Time   `json:"acquire_time" spanner:"acquire_time"~`
    ExpiresTime     spanner.NullTime `json:"expires_time" spanner:"expires_time"~`
    Visible         bool        `json:"visible"~`
}

type Player struct {
    PlayerUUID      string      `json:"playerUUID" binding:"required,uuid4"~`
    Updated         time.Time   `json:"updated"~`
    AccountBalance  big.Rat     `json:"account_balance" spanner:"account_balance"~`
    CurrentGame     string      `json:"current_game" binding:"omitempty,uuid4"~`
    spanner:"current_game"~`
}

type PlayerLedger struct {
    PlayerUUID      string      `json:"playerUUID" binding:"required,uuid4"~`
    Amount          big.Rat     `json:"amount"~`
    GameSession     string      `json:"game_session" spanner:"game_session"~`
    Source          string      `json:"source"~`
}

```

如要建立交易訂單，請向 API 端點 **/trades/sell** 發出 POST 要求。必要資訊包括要販售的 player_item 的 **playerItemUUID**、**lister** 和 **list_price**。

選擇 [Spanner 突變](#) 來建立交易訂單，並將 player_item 標示為「不可見」。這樣做可避免賣家重複刊登商品。

```

func (o *TradeOrder) Create(ctx context.Context, client spanner.Client) error {
    // insert into spanner
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        // get the Item to be listed
        pi, err := GetPlayerItem(ctx, txn, o.Lister, o.PlayerItemUUID)
        if err != nil {
            return err
        }

        // Set expires to 1 day by default
        if o.Expires.IsZero() {
            currentTime := time.Now()
            o.Expires = currentTime.Add(time.Hour * 24)
        }

        // Item is not visible or expired, so it can't be listed. That's an error
        if !validateSellOrder(pi) {
            errorMsg := fmt.Sprintf("Item (%s, %s) cannot be listed.", o.Lister,
o.PlayerItemUUID)
            return errors.New(errorMsg)
        }

        // Initialize order values
        o.OrderUUID = generateUUID()
        o.Active = true // TODO: Have to set this by default since testing with emulator does
not support 'DEFAULT' schema option

        // Insert the order
        var m []*spanner.Mutation
        cols := []string{"orderUUID", "playerItemUUID", "lister", "list_price", "trade_type",
"expires", "active"}
        m = append(m, spanner.Insert("trade_orders", cols, []interface{}{o.OrderUUID,
o.PlayerItemUUID, o.Lister, o.ListPrice, "sell", o.Expires, o.Active}))

        // Mark the item as invisible
        cols = []string{"playerUUID", "playerItemUUID", "visible"}
        m = append(m, spanner.Update("player_items", cols, []interface{}{o.Lister,
o.PlayerItemUUID, false}))

        txn.BufferWrite(m)
        return nil
    })

    if err != nil {
        return err
    }

    // return empty error on success
    return nil
}

```

實際建立訂單前，系統會驗證 **PlayerItem**，確保可供販售。這主要是指 **PlayerItem** 對玩家可見，且尚未過期。

```
// Validate that the order can be placed: Item is visible and not expired
func validateSellOrder(pi PlayerItem) bool {
    // Item is not visible, can't be listed
    if !pi.Visible {
        return false
    }

    // item is expired. can't be listed
    if !pi.ExpiresTime.IsNull() && pi.ExpiresTime.Time.Before(time.Now()) {
        return false
    }

    // All validation passed. Item can be listed
    return true
}
```

注意：本範例應用程式中產生的所有項目都不會過期。處理過期項目會是另一個服務的背景程序，但這項功能尚未實作。

如要購買，請向 **/trades/buy** 端點發出 PUT 要求。必要資訊為 **orderUUID** 和買家 (即進行購買的玩家 UUID)。

由於這項複雜度和變更數量，突變再次被選中來購買訂單。下列作業會在單一讀寫交易中完成：

- 驗證訂單是否尚未填寫且未過期，因此可以填寫。

```
// Validate that the order can be filled: Order is active and not expired
func validatePurchase(o TradeOrder) bool {
    // Order is not active
    if !o.Active {
        return false
    }

    // order is expired. can't be filled
    if !o.Expires.IsZero() && o.Expires.Before(time.Now()) {
        return false
    }

    // All validation passed. Order can be filled
    return true
}
```

- 擷取買家資訊，並驗證買家是否可以購買商品。也就是說，買家不得與賣家相同，且買家必須有足夠的資金。

```
// Validate that a buyer can buy this item.
func validateBuyer(b Player, o TradeOrder) bool {
    // Lister can't be the same as buyer
    if b.PlayerUUID == o.Lister {
        return false
    }

    // Big.Rat returns -1 if Account_balance is less than price
    if b.AccountBalance.Cmp(&o.ListPrice) == -1 {
        return false
    }

    return true
}
```

- 將訂單的 *list_price* 加入刊登者的帳戶餘額，並提供相符的分類帳項目。

```
// models/trade_order.go
// Buy an order
func (o *TradeOrder) Buy(ctx context.Context, client spanner.Client) error {
*snip*
    // Update seller's account balance
    lister.UpdateBalance(ctx, txn, o.ListPrice)
*snip*
}

// models/players.go
// Update a player's balance, and add an entry into the player ledger
func (p *Player) UpdateBalance(ctx context.Context, txn *spanner.ReadWriteTransaction,
newAmount big.Rat) error {
    // This modifies player's AccountBalance, which is used to update the player entry
    p.AccountBalance.Add(&p.AccountBalance, &newAmount)

    txn.BufferWrite([]*spanner.Mutation{
        spanner.Update("players", []string{"playerUUID", "account_balance"}, []interface{}{
            {p.PlayerUUID, p.AccountBalance}}),
        spanner.Insert("player_ledger_entries", []string{"playerUUID", "amount",
"game_session", "source", "entryDate"},
            []interface{}{p.PlayerUUID, newAmount, p.CurrentGame, "tradepest",
spanner.CommitTimestamp}),
    })
    return nil
}
```

- 從買家帳戶餘額中扣除訂單的 *list_price*，並建立相符的分類帳項目。

```
// Update buyer's account balance
negAmount := o.ListPrice.Neg(&o.ListPrice)
buyer.UpdateBalance(ctx, txn, *negAmount)
```

- 將 `player_item` 移至新玩家，方法是將遊戲項目的新執行個體插入 **PlayerItems** 表格，並提供遊戲和買家詳細資料，然後移除賣家的項目執行個體。

```
// models/player_items.go
// Move an item to a new player, removes the item entry from the old player
func (pi *PlayerItem) MoveItem(ctx context.Context, txn *spanner.ReadWriteTransaction,
toPlayer string) error {
    fmt.Printf("Buyer: %s", toPlayer)
    txn.BufferWrite([]*spanner.Mutation{
        spanner.Insert("player_items", []string{"playerItemUUID", "playerUUID", "itemUUID",
"price", "source", "game_session"},
        []interface{}{pi.PlayerItemUUID, toPlayer, pi.ItemUUID, pi.Price, pi.Source,
pi.GameSession}),
        spanner.Delete("player_items", spanner.Key{pi.PlayerUUID, pi.PlayerItemUUID}),
    })

    return nil
}
```

- 更新「訂單」項目，指出商品已填寫完畢，不再有效。

綜合來看，**Buy** 函式如下所示：

```

// Buy an order
func (o *TradeOrder) Buy(ctx context.Context, client spanner.Client) error {
    // Fulfil the order
    _, err := client.ReadWriteTransaction(ctx, func(ctx context.Context, txn
*spanner.ReadWriteTransaction) error {
        // Get Order information
        err := o.getOrderDetails(ctx, txn)
        if err != nil {
            return err
        }

        // Validate order can be filled
        if !validatePurchase(*o) {
            errorMsg := fmt.Sprintf("Order (%s) cannot be filled.", o.OrderUUID)
            return errors.New(errorMsg)
        }

        // Validate buyer has the money
        buyer := Player{PlayerUUID: o.Buyer}
        err = buyer.GetBalance(ctx, txn)
        if err != nil {
            return err
        }

        if !validateBuyer(buyer, *o) {
            errorMsg := fmt.Sprintf("Buyer (%s) cannot purchase order (%s).",
buyer.PlayerUUID, o.OrderUUID)
            return errors.New(errorMsg)
        }

        // Move money from buyer to seller (which includes ledger entries)
        var m []*spanner.Mutation
        lister := Player{PlayerUUID: o.Lister}
        err = lister.GetBalance(ctx, txn)
        if err != nil {
            return err
        }

        // Update seller's account balance
        lister.UpdateBalance(ctx, txn, o.ListPrice)

        // Update buyer's account balance
        negAmount := o.ListPrice.Neg(&o.ListPrice)
        buyer.UpdateBalance(ctx, txn, *negAmount)

        // Move item from seller to buyer, mark item as visible.
        pi, err := GetPlayerItem(ctx, txn, o.Lister, o.PlayerItemUUID)
        if err != nil {
            return err
        }
        pi.GameSession = buyer.CurrentGame
    })
}

```

```

    // Moves the item from lister (current pi.PlayerUUID) to buyer
    pi.MoveItem(ctx, txn, o.Buyer)

    // Update order information
    cols := []string{"orderUUID", "active", "filled", "buyer", "ended"}
    m = append(m, spanner.Update("trade_orders", cols, []interface{}{o.OrderUUID, false,
true, o.Buyer, time.Now()}))

    txn.BufferWrite(m)
    return nil
})

if err != nil {
    return err
}

// return empty error on success
return nil
}

```

根據預設，服務會使用環境變數設定。請參閱 [./src/golang/tradepost-service/config/config.go](#) 檔案的相關部分。

```

func NewConfig() (Config, error) {
    *snip*
    // Server defaults
    viper.SetDefault("server.host", "localhost")
    viper.SetDefault("server.port", 8083)

    // Bind environment variable override
    viper.BindEnv("server.host", "SERVICE_HOST")
    viper.BindEnv("server.port", "SERVICE_PORT")
    viper.BindEnv("spanner.project_id", "SPANNER_PROJECT_ID")
    viper.BindEnv("spanner.instance_id", "SPANNER_INSTANCE_ID")
    viper.BindEnv("spanner.database_id", "SPANNER_DATABASE_ID")

    *snip*

    return c, nil
}

```

您可以看到，預設行為是在 `localhost:8083` 上執行服務，以避免與其他服務發生衝突。

有了這些資訊，現在可以執行 tradepost 服務。

執行服務

執行服務會在通訊埠 8083 上建立服務。這項服務與項目服務有許多相同的依附元件，因此不會下載新的依附元件。

```

cd ~/spanner-gaming-sample/src/golang/tradepost-service
go run . &

```

注意：下載 golang 依附元件可能需要幾分鐘。

指令輸出：

```
[GIN-debug] [WARNING] Creating an Engine instance with the Logger and Recovery middleware already attached.

[GIN-debug] [WARNING] Running in "debug" mode. Switch to "release" mode in production.
- using env:    export GIN_MODE=release
- using code:   gin.SetMode(gin.ReleaseMode)

[GIN-debug] GET    /trades/player_items    --> main.getPlayerItem (4 handlers)
[GIN-debug] POST   /trades/sell             --> main.createOrder (4 handlers)
[GIN-debug] GET    /trades/open             --> main.getOpenOrder (4 handlers)
[GIN-debug] PUT    /trades/buy              --> main.purchaseOrder (4 handlers)
[GIN-debug] Listening and serving HTTP on localhost:8083
```

發布項目

發出 GET 要求來擷取要販售的 *PlayerItem*，測試服務：

```
curl http://localhost:8083/trades/player_items
```

指令輸出：

```
{
  "PlayerUUID": "b74cc194-87b0-4a55-a67f-0f0742ef6352",
  "PlayerItemUUID": "a42b1899-4509-4fce-9958-265d2a2838a0",
  "Price": "3.14"
}
```

****注意：****系統會在下個步驟使用這項商品資訊建立銷售訂單

現在，請呼叫 **/trades/sell** 端點，上架銷售商品

```
curl http://localhost:8083/trades/sell \
  --include \
  --header "Content-Type: application/json" \
  --request "POST" \
  --data '{"lister": "<PlayerUUID>", "playerItemUUID": "<PlayerItemUUID>", "list_price": "<some price higher than item's price>"}'
```

注意：請務必填入上一個步驟中 GET 要求的 *PlayerUUID*、*PlayerItemUUID* 和 *list_price* 資料。

指令輸出：

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Date: <Date>
Content-Length: 38

"282ea691-b956-4c4c-95ff-f461d6415651"
```

摘要

在這個步驟中，您已部署 **trade-post-service** 來處理建立賣單。這項服務也負責處理購買這些訂單的功能。

後續步驟

服務已開始運作，現在可以模擬玩家在交易站買賣球員！

5. 開始交易

現在項目和交易所服務都已開始執行，您可以使用提供的蝗蟲產生器產生負載。

Locust 提供網頁介面來執行產生器，但本實驗室會使用指令列 (*-headless* 選項)。

生成遊戲道具

首先，請產生項目。[./generators/item_generator.py](#) 檔案包含一項工作，可建立名為隨機字串的遊戲項目，以及隨機價格值：

```
# Generate random items
class ItemLoad(HttpUser):
    def generateItemName(self):
        return ''.join(random.choices(string.ascii_lowercase + string.digits, k=32))

    def generateItemValue(self):
        return str(decimal.Decimal(random.randrange(100, 10000))/100)

    @task
    def createItem(self):
        headers = {"Content-Type": "application/json"}
        data = {"item_name": self.generateItemName(), "item_value": self.generateItemValue()}

        self.client.post("/items", data=json.dumps(data), headers=headers)
```

下列指令會呼叫 *item_generator.py* 檔案，產生 10 秒的遊戲項目 (*t=10s*)：

```
cd ~/spanner-gaming-sample
locust -H http://127.0.0.1:8082 -f ./generators/item_generator.py --headless -u=1 -r=1 -t=10s
```

注意：執行產生器時，locust 指令會輸出所發出要求的統計資料，包括要求數、失敗數和延遲時間資訊。

指令輸出：

```

*snip*
/INFO/locust.main: --run-time limit reached. Stopping Locust
/INFO/locust.main: Shutting down (exit code 0)
Name                                                    # reqs
# fails | Avg      Min      Max  Median | req/s failures/s
-----
POST /items                                                    606
0(0.00%) | 16      12      161    15 | 60.61    0.00
-----
Aggregated                                                    606
0(0.00%) | 16      12      161    15 | 60.61    0.00
-----

Response time percentiles (approximated)
Type      Name
50%      66%      75%      80%      90%      95%      98%      99%      99.9% 99.99% 100% # reqs
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
POST      /items
15      16      16      17      18      19      21      34      160    160    160    606
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
None      Aggregated
15      16      16      17      18      19      21      34      160    160    160    606
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

```

玩家取得物品和金錢

接著，讓玩家取得物品和金錢，以便在交易站上進行交易。為此，[./generators/game_server.py](#) 檔案會提供工作，以擷取要指派給玩家的遊戲道具，以及隨機金額的貨幣。

```

# Players generate items and money at 5:2 ratio. We don't want to devalue the currency!
class GameLoad(HttpUser):
    def on_start(self):
        self.getItems()

    def getItems(self):
        headers = {"Content-Type": "application/json"}
        r = requests.get(f"{self.host}/items", headers=headers)

        global itemUUIDs
        itemUUIDs = json.loads(r.text)

    def generateAmount(self):
        return str(round(random.uniform(1.01, 49.99), 2))

    @task(2)
    def acquireMoney(self):
        headers = {"Content-Type": "application/json"}

        # Get a random player that's part of a game, and update balance
        with self.client.get("/players", headers=headers, catch_response=True) as response:
            try:
                data = {"playerUUID": response.json()["playerUUID"], "amount":
self.generateAmount(), "source": "loot"}
                self.client.put("/players/balance", data=json.dumps(data), headers=headers)
            except json.JSONDecodeError:
                response.failure("Response could not be decoded as JSON")
            except KeyError:
                response.failure("Response did not contain expected key 'playerUUID'")

    @task(5)
    def acquireItem(self):
        headers = {"Content-Type": "application/json"}

        # Get a random player that's part of a game, and add an item
        with self.client.get("/players", headers=headers, catch_response=True) as response:
            try:
                itemUUID = itemUUIDs[random.randint(0, len(itemUUIDs)-1)]
                data = {"playerUUID": response.json()["playerUUID"], "itemUUID": itemUUID,
"source": "loot"}
                self.client.post("/players/items", data=json.dumps(data), headers=headers)
            except json.JSONDecodeError:
                response.failure("Response could not be decoded as JSON")
            except KeyError:
                response.failure("Response did not contain expected key 'playerUUID'")

```

注意：只有目前在「開放遊戲」中的玩家可以取得物品和金錢。如果沒有任何開放遊戲，您必須使用 **match_server.py** 產生器產生遊戲。

這個指令可讓玩家在 60 秒內取得物品和金錢：

```
locust -H http://127.0.0.1:8082 -f game_server.py --headless -u=1 -r=1 -t=60s
```

指令輸出：

```
*snip*
dev-machine/INFO/locust.main: --run-time limit reached. Stopping Locust
dev-machine/INFO/locust.main: Shutting down (exit code 0)
Name                                     # reqs
# fails | Avg      Min      Max  Median | req/s failures/s
-----|-----
GET /players                                     231
0(0.00%) | 14       9       30    13 | 23.16    0.00
PUT /players/balance                             53
0(0.00%) | 33      30      39    34 |  5.31    0.00
POST /players/items                             178
0(0.00%) | 26      22      75    26 | 17.85    0.00
-----|-----
Aggregated                                     462
0(0.00%) | 21       9      75    23 | 46.32    0.00

Response time percentiles (approximated)
Type      Name
50%      66%      75%      80%      90%      95%      98%      99%      99.9% 99.99% 100% # reqs
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
GET      /players
13       16       17       17       19       20       21       23       30      30      30      231
PUT      /players/balance
34       34       35       35       36       37       38       40       40      40      40      53
POST     /players/items
26       27       27       27       28       29       34       53       76      76      76      178
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
None     Aggregated
23       26       27       27       32       34       36       37       76      76      76      462
```

玩家在交易站買賣物品

玩家現在有物品和購買物品的資金，可以開始使用交易站！

[./generators/trading_server.py](#) 生成器檔案提供建立銷售訂單及完成這些訂單的任務。

```

# Players can sell and buy items
class TradeLoad(HttpUser):
    def itemMarkup(self, value):
        f = float(value)
        return str(f*1.5)

    @task
    def sellItem(self):
        headers = {"Content-Type": "application/json"}

        # Get a random item
        with self.client.get("/trades/player_items", headers=headers, catch_response=True) as response:
            try:
                playerUUID = response.json()["PlayerUUID"]
                playerItemUUID = response.json()["PlayerItemUUID"]
                list_price = self.itemMarkup(response.json()["Price"])

            # Currently don't have any items that can be sold, retry
            if playerItemUUID == "":
                raise RescheduleTask()

            data = {"lister": playerUUID, "playerItemUUID": playerItemUUID, "list_price":
list_price}

            self.client.post("/trades/sell", data=json.dumps(data), headers=headers)
        except json.JSONDecodeError:
            response.failure("Response could not be decoded as JSON")
        except KeyError:
            response.failure("Response did not contain expected key 'playerUUID'")

    @task
    def buyItem(self):
        headers = {"Content-Type": "application/json"}

        # Get a random item
        with self.client.get("/trades/open", headers=headers, catch_response=True) as response:
            try:
                orderUUID = response.json()["OrderUUID"]
                buyerUUID = response.json()["BuyerUUID"]

                # Currently don't have any buyers that can fill the order, retry
                if buyerUUID == "":
                    raise RescheduleTask()

                data = {"orderUUID": orderUUID, "buyer": buyerUUID}
                self.client.put("/trades/buy", data=json.dumps(data), headers=headers)
            except json.JSONDecodeError:
                response.failure("Response could not be decoded as JSON")

```

```
except KeyError:
    response.failure("Response did not contain expected key 'playerUUID'")
```

這個指令可讓玩家列出已取得的商品供販售，其他玩家則可在 10 秒內購買這些商品：

```
locust -H http://127.0.0.1:8083 -f ./generators/trading_server.py --headless -u=1 -r=1 -t=10s
```

指令輸出：

```
*snip*
Name                                                    # reqs
# fails |      Avg      Min      Max  Median |    req/s failures/s
-----|-----
PUT /trades/buy                                                    20
5(25.00%) |      43      10      78      43 |    2.07    0.52
GET /trades/open                                                    20
0(0.00%) |     358       7    971    350 |    2.07    0.00
GET /trades/player_items                                            20
0(0.00%) |      49      35    113      41 |    2.07    0.00
POST /trades/sell                                                    20
0(0.00%) |      29      21    110      24 |    2.07    0.00
-----|-----
Aggregated                                                        80
5(6.25%) |     120       7    971      42 |    8.29    0.52

Response time percentiles (approximated)
Type      Name
50%      66%      75%      80%      90%      95%      98%      99%      99.9% 99.99% 100% # reqs
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
PUT      /trades/buy
43      45      49      50      71      78      78      78      78      78      78      20
GET      /trades/open
360     500     540     550     640     970     970     970     970     970     970     20
GET      /trades/player_items
43      55      57      59      72      110     110     110     110     110     110     20
POST     /trades/sell
24      25      25      27      50      110     110     110     110     110     110     20
-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
None     Aggregated
42      50      71     110     440     550     640     970     970     970     970     80
```

摘要

在這個步驟中，您模擬玩家註冊玩遊戲，然後使用媒合服務模擬玩家玩遊戲。這些模擬作業運用 Locust Python 架構，向服務的 REST API 發出要求。

您可以隨意修改建立玩家和玩遊戲所花的時間，以及並行使用者人數 (-u)。

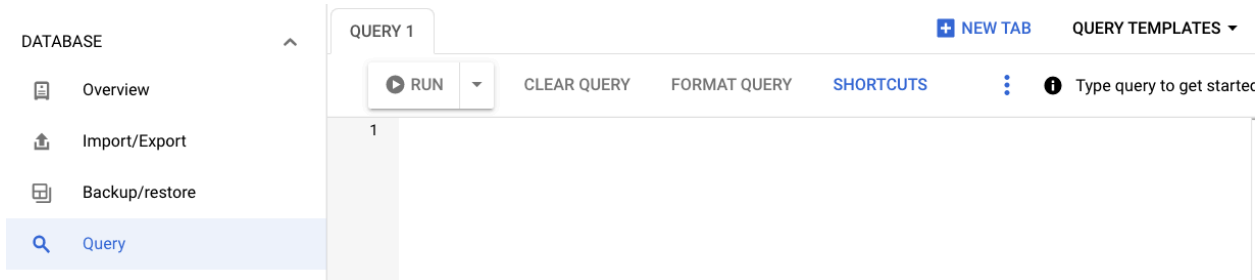
後續步驟

模擬完成後，您會想查詢 **Spanner**，查看各種統計資料。

6. 擷取交易統計資料

我們已模擬玩家取得金錢和物品，然後在交易站出售這些物品，現在來查看一些統計資料。

如要執行這項操作，請使用 Cloud Console 向 Spanner 發出查詢要求。



查看未結與已完成的交易訂單

在交易平台購買 **TradeOrder** 時，系統會更新 *filled* 中繼資料欄位。

這項查詢可讓您查看未結案的訂單數量，以及已完成的訂單數量：

```
-- Open vs Filled Orders
SELECT Type, NumTrades FROM
(SELECT "Open Trades" as Type, count(*) as NumTrades FROM trade_orders WHERE active=true
UNION ALL
SELECT "Filled Trades" as Type, count(*) as NumTrades FROM trade_orders WHERE filled=true
)
```

結果：

類型	NumTrades
開啟交易	159
成交交易	454

注意：開放交易與已完成交易的確切結果會有所不同。

查看玩家帳戶餘額和項目數量

如果玩家的 *current_game* 欄已設定，表示玩家正在玩遊戲。否則表示他們目前沒有在玩遊戲。

如要找出目前正在玩遊戲且擁有最多項目的前 10 名玩家，以及他們的 *account_balance*

，請使用以下查詢：

```
SELECT playerUUID, account_balance, (SELECT COUNT(*) FROM player_items WHERE
playerUUID=p.PlayerUUID) AS numItems, current_game
FROM players AS p
WHERE current_game IS NOT NULL
ORDER BY numItems DESC
LIMIT 10;
```

結果：

playerUUID	account_balance	numItems	current_game
04d14288-a7c3-4515-9296-44eeae184d6d	1396.55	201	79f24cd4-102d-4d7d-bbe8-be69d364ebbf
03836948-d591-4967-a8a8-80506454916d	2005.085	192	053c353e-e56d-4b2e-9431-76eedf58bea5
snip	snip	snip	snip

注意：每位玩家的實際結果會有所不同。

摘要

在這個步驟中，您使用 Cloud 控制台查詢 Spanner，查看了玩家和交易訂單的各種統計資料。

後續步驟

接著，我們來清除主題！

步驟 7 · 約 2 分鐘

7. 正在清除所用資源

玩完 Spanner 後，我們需要清理遊樂場。幸好這個步驟很簡單，只要前往 Cloud 控制台的 [Cloud Spanner 區段](#)，然後刪除我們為這個程式碼研究室建立的執行個體即可。

步驟 8 · 約 2 分鐘

8. 恭喜！

恭喜！您已成功在 Spanner 上部署範例遊戲

後續步驟

在本實驗室中，您已完成設定兩項服務，分別用於處理遊戲道具生成作業，以及玩家取得要在交易站販售的道具。

這些程式碼範例應有助於您進一步瞭解，交易中的 Cloud Spanner 一致性如何適用於 [DML](#) 和 [Spanner 突變](#)。

歡迎使用提供的產生器，探索如何擴充 Spanner。
